

Day 19 Quiz– Q19 – EF Core Code First, Service Layer, and Repository Pattern

1. Which command creates a new EF Core migration?

- A. dotnet ef migration create InitialCreate
- B. dotnet ef migrations add InitialCreate
- C. dotnet ef database add InitialCreate
- D. dotnet ef model add InitialCreate

Answer: B.

2. Which command applies pending migrations to the database?

- A. dotnet ef migrations update
- B. dotnet ef database migrate
- C. dotnet ef database update
- D. dotnet ef update database

Answer: C.

3. Consider the following entity:

```
public class Product
{
    public int ProductId { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

By convention, EF Core identifies ProductId as:

- A. Foreign key
- B. Primary key
- C. Alternate key
- D. Composite key

Answer: B.

4. Fill in the blank:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.Name)
        ._____ (100);
}
```

Which method specifies the maximum length?

- A. MaxLength
- B. HasMaxLength
- C. MaximumLength
- D. SetMaxLength

Answer: B.

5. Which configuration correctly establishes a one-to-many relationship?

```
modelBuilder.Entity<Category>()  
    .HasMany(c => c.Products)  
    ._____ (p => p.Category);
```

- A. HasOne
- B. WithMany
- C. WithOne
- D. HasMany

Answer: B.

6. Consider:

```
public class Product  
{  
    public int ProductId { get; set; }  
    public int CategoryId { get; set; }  
    public Category Category { get; set; }  
}
```

What does CategoryId normally represent?

- A. Primary key of Product
- B. Foreign key to Category
- C. Composite key
- D. Shadow property

Answer: B.

7. Which registration is appropriate for an EF Core DbContext in ASP.NET Core?

- A. services.AddSingleton<AppDbContext>()
- B. services.AddTransient<AppDbContext>()
- C. services.AddDbContext<AppDbContext>(...)
- D. services.AddStatic<AppDbContext>()

Answer: C.

8. What happens when the following code is executed?

```
var products = await _context.Products  
    .Where(p => p.Price > 1000)  
    .ToListAsync();
```

- A. All records are loaded first and filtered in memory
- B. EF Core generally translates the query into SQL
- C. The database table is deleted
- D. A migration is created

Answer: B.

9. Consider:

```
modelBuilder.Entity<Product>()  
    .Property(p => p.Name)  
    .IsRequired();
```

What does this configuration indicate?

- A. Product name must be unique
- B. Product name cannot be null
- C. Product name must contain only numbers
- D. Product name becomes the primary key

Answer: B.

10. Fill in the code:

```
var product = await _context.Products  
    .FirstOrDefaultAsync(p => p.ProductId == id);  
  
if (product == null)  
{  
    return _____;  
}
```

If this code is inside a service and the method should return null, what should replace the blank?

- A. empty
- B. default
- C. null
- D. NotFound

Answer: C.

11. Consider:

```
public interface IProductService
```

```

{
    Task<ProductDto?> GetByIdAsync(int id);
    Task<IEnumerable<ProductDto>> GetAllAsync();
}

```

Which class should normally implement this interface?

- A. ProductController
- B. ProductRepository
- C. ProductService
- D. ProductDbContext

Answer: C.

12. Fill in the code:

```

public class ProductService : IProductService
{
    private readonly IProductRepository _repository;
    public ProductService(_____ repository)
    {
        _repository = repository;
    }
}

```

- A. ProductRepository
- B. IProductRepository
- C. DbContext
- D. IProductService

Answer: B.

13. Consider:

```

public async Task<ProductDto> CreateAsync(ProductDto dto)
{
    if (dto.Price <= 0)
        throw new ArgumentException("Invalid price");

    // save product
}

```

What is this code demonstrating?

- A. Authentication
- B. Business-rule validation in the Service Layer
- C. Database migration
- D. Repository implementation

Answer: B.

14. Consider:

```
[HttpGet("{id}")]
```

```
public async Task<ActionResult> Get(int id)
{
    var product = await _service.GetByIdAsync(id);
    if (product == null)
        return NotFound();
    return Ok(product);
}
```

What is the Service Layer responsible for here?

- A. Deciding HTTP status code
- B. Retrieving the requested product
- C. Generating route metadata
- D. Parsing the HTTP URL

Answer: B.

15. Fill in the code:

```
public async Task<Product?> GetByIdAsync(int id)
{
    return await _context.Products
        .FirstOrDefaultAsync(p => p._____);
}
```

- A. Id == id
- B. ProductId == id
- C. ProductID = id
- D. id = ProductId

Answer: B.

16. Which query returns a single product or null?

- A. `_context.Products.ToListAsync();`
- B. `_context.Products.FirstOrDefault(p => p.ProductId == id);`

- C. `_context.Products.AsEnumerableAsync();`
- D. `_context.Products.WhereAsync(p => p.ProductId == id);`

Answer: B.

17. Fill in the missing code:

```
public class ProductRepository : IProductRepository
{
    private readonly AppDbContext _context;

    public ProductRepository(AppDbContext context)
    {
        _____ = context;
    }
}
```

- A. context
- B. `_context`
- C. AppDbContext
- D. IProductRepository

Answer: B.

18. Which implementation best demonstrates the dependency flow?

- A. Controller → DbContext → Repository → Service
- B. Controller → Repository → Service → DbContext
- C. Controller → Service → Repository → DbContext
- D. Controller → DbContext → Service → Repository

- A. A
- B. B
- C. C
- D. D

Answer: C.

19. Scenario-based / Fill-in-the-code:

You have the following architecture:

ProductController

↓

IProductService

↓

IProductRepository



AppDbContext



SQL Server

The requirement is:

When creating a product, the product name must be unique and the price must be greater than zero.

Which is the **best design**?

- A. Put both rules inside ProductRepository
- B. Put both rules inside ProductController
- C. Put business rules in ProductService and database persistence in ProductRepository
- D. Put both rules inside AppDbContext

Answer: C.

20. Consider the following Service:

```
public async Task<Product> CreateAsync(Product product)
{
    if (product.Price <= 0)
        throw new ArgumentException("Price must be greater than zero");
    await _repository.AddAsync(product);
    return product;
}
```

The repository contains:

```
public async Task AddAsync(Product product)
{
    await _context.Products.AddAsync(product);
}
```

The API returns success, but the record is **not persisted to SQL Server**.

What is the most likely missing operation?

- A. ToListAsync() in the Service
- B. SaveChangesAsync() in the persistence/unit-of-work flow
- C. AsNoTracking() in the Repository
- D. FirstOrDefaultAsync() in the Controller

Answer: B.